

Query Optimization: Examples of Transformations

1 Illustrating Transformation Rules

Using a concrete example from a university database, we can demonstrate how equivalence rules are applied to generate equivalent expressions that may have lower execution costs.

1.1 Relation Schemas

The following schemas are used in our examples:

- `instructor`(ID, name, dept_name, salary)
- `teaches`(ID, course_id, sec_id, semester, year)
- `course`(course_id, title, dept_name, credits)

1.2 Initial Transformation Example

Consider the expression to find the names of instructors and titles of courses for the "Music" department:

$$\Pi_{name, title}(\sigma_{dept_name="Music"}(instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$$

Using **Rule 7.a** (Distribution of σ over $\bowtie$), we can transform this into:

$$\Pi_{name, title}((\sigma_{dept_name="Music"}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id, title}(course)))$$

Heuristic Benefit

This transformation generates smaller intermediate relations by filtering the `instructor` relation *before* performing the join.

2 Multiple Rule Transformations

When a query is more complex, multiple rules can be applied one after another. Suppose we restrict the query to instructors who taught a course in 2017.

2.1 Step-by-Step Transformation

1. Initial Query:

$$\Pi_{name, title}(\sigma_{dept_name="Music" \wedge year=2017}(instructor \bowtie (teaches \bowtie \Pi_{course_id, title}(course))))$$

2. Apply Rule 6.a (Associativity of Natural Join): We transform $instructor \bowtie (teaches \bowtie \Pi)$ into $(instructor \bowtie teaches) \bowtie \Pi$:

$$\Pi_{name,title}(\sigma_{dept_name="Music" \wedge year=2017}((instructor \bowtie teaches) \bowtie \Pi_{course_id,title}(course)))$$

3. Apply Rule 7.a (Pushing Selection): Move the selection inside the join:

$$\Pi_{name,title}((\sigma_{dept_name="Music" \wedge year=2017}(instructor \bowtie teaches)) \bowtie \Pi_{course_id,title}(course))$$

4. Apply Rule 1 (Cascade of σ): Break the selection into two parts:

$$\sigma_{dept_name="Music"}(\sigma_{year=2017}(instructor \bowtie teaches))$$

5. Apply Rule 7.a Again: Further optimize by pushing selections to individual relations:

$$\sigma_{dept_name="Music"}(instructor) \bowtie \sigma_{year=2017}(teaches)$$

2.2 Expression Tree Visualization

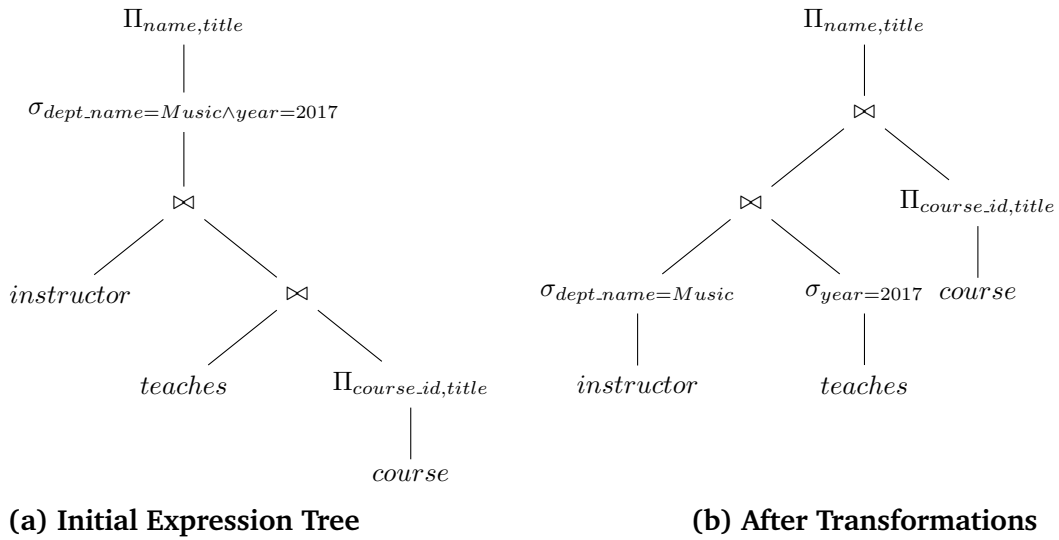


Figure 1: Multiple transformations applied to an expression tree.

3 Projection Pushing

To further reduce the size of intermediate results, we can push projections to eliminate unneeded attributes early.

Using **Rules 8.a and 8.b**, we can modify the expression:

$$\Pi_{name,title}((\Pi_{name,course_id}((\sigma_{dept_name="Music"}(instructor)) \bowtie teaches)) \bowtie \Pi_{course_id,title}(course))$$

Important

The projection $\Pi_{name, course_id}$ reduces the intermediate join results to only the columns necessary for subsequent steps.

4 Join Ordering

A good ordering of join operations is critical for reducing the size of temporary results; hence, most query optimizers pay significant attention to the join order.

4.1 Cost Analysis of Join Associativity

As mentioned in **Rule 6.a**, the natural-join operation is associative:

$$(r_1 \bowtie r_2) \bowtie r_3 \equiv r_1 \bowtie (r_2 \bowtie r_3)$$

Case Study: Music Department Query

Consider the expression:

$$\Pi_{name, title}((\sigma_{dept_name = \text{“Music”}}(instructor)) \bowtie teaches \bowtie \Pi_{course_id, title}(course))$$

We can evaluate this in two primary sequences:

- **Suboptimal Order:** $(teaches \bowtie course_id)$ first.
 - Result is a **large relation** containing every course taught in the university.
- **Optimal Order:** $(instructor \bowtie teaches)$ first.
 - Result is a **small relation** because only a few instructors teach in the Music department.

Conclusion: Strategic join ordering minimizes the memory and CPU required for intermediate storage.

4.2 Avoiding Cartesian Products

Using **Rule 5** (Commutativity) and **Rule 6** (Associativity), optimizers can detect and avoid implicit Cartesian products.

Caution: Cartesian Product Risk

Consider the expression: $(instructor \bowtie \Pi_{course_id, title}(course)) \bowtie teaches$.

- **The Issue:** No common attributes between *instructor* and *course*.
- **The Result:** A massive temporary relation of size $a \times b$ tuples.

Optimizer transforms this to: $(instructor \bowtie teaches) \bowtie course$.

5 Enumeration of Equivalent Expressions

Query optimizers systematically generate equivalent expressions using a procedure like the one below.

5.1 The Generation Algorithm

Procedure: genAllEquivalent(E)

```
EQ = {E}
repeat
    Match each expression  $E_i$  in EQ with each equivalence rule  $R_j$ 
    if any subexpression  $e_i$  of  $E_i$  matches one side of  $R_j$ 
        Create a new expression  $E'$  by transforming  $e_i$  to match  $R_j$ 's other side
        Add  $E'$  to EQ if it is not already present
until no new expression can be added to EQ
```

5.2 Optimization Key Ideas

1. **Shared Subexpressions:** Use pointers to identical subtrees to save space.
2. **Cost-based Pruning:** Avoid generating or exploring expressions that are clearly sub-optimal based on initial cost estimates.